

Academic Year 2025 – 2026
Research Unit: Universidad Loyola
Team(s): Máster en Inteligencia Artificial
Supervisor(s): Bernardo Ronquillo Japón



MLOps

LAB3: Continuous Training: app-iris-ct

Ignacio González Peris

6 de marzo de 2026

Abstract

En este laboratorio se ha implementado un sistema de *Continuous Training (CT)* sobre un servidor de clasificación de flores Iris. El objetivo es que el modelo pueda reentrenarse con nuevos datos en producción, evaluar si el nuevo modelo mejora al anterior, y activarlo automáticamente solo si supera un criterio de calidad configurable, sin necesidad de reiniciar el servidor.

Keywords: MLOps, Continuous Training, FastAPI, LogisticRegression, Quality Gate, Model Versioning

Contents

1 Descripción general	3
2 Arquitectura del sistema	3
3 Flujo de Continuous Training	3
3.1 Bootstrap	3
3.2 Ciclo de reentrenamiento	3
4 Razonamiento previo: elección de política por escenario	4
4.1 Escenario 1: Detección de fraude bancario	4
4.2 Escenario 2: Recomendación de películas	4
4.3 Escenario 3: Diagnóstico médico	4
4.4 Diseño previo en pseudocódigo	4
5 Políticas de activación (ejercicio de programación)	5
5.1 any_improvement (por defecto)	5
5.2 min_delta	5
5.3 per_class_f1	5
6 Ejercicio actual de la entrega LAB3	6
6.1 Ejecución del flujo completo	6
6.2 Análisis del historial	6
6.3 Incremental vs. desde cero	7
7 Preguntas de reflexión	7
7.1 Reproducibilidad: si se borra models/ y se reinicia el servidor, ¿se obtiene el mismo modelo?	7
7.2 Edge case: ¿qué ocurre si todas las muestras enviadas son de la misma clase?	8
7.3 ¿Por qué registrar los modelos rechazados en el historial?	8
7.4 Quality gate: $\geq$ o $>$?	8

1 Descripción general

En este laboratorio se ha implementado un sistema de **Continuous Training (CT)** sobre un servidor de clasificación de flores Iris. El objetivo es que el modelo pueda reentrenarse con nuevos datos en producción, evaluar si el nuevo modelo mejora al anterior, y activarlo automáticamente solo si supera un criterio de calidad configurable, sin necesidad de reiniciar el servidor.

2 Arquitectura del sistema

El servidor está construido con **FastAPI** y expone los siguientes endpoints:

Endpoint	Método	Función
/health	GET	Estado del servidor y versión del modelo activo
/predict	POST	Clasificación de una flor con el modelo activo
/train	POST	Reentrenamiento con nuevas muestras etiquetadas
/model/info	GET	Metadata del modelo activo e historial completo
/model/history	DELETE	Resetea el historial y restaura el modelo base

El modelo utilizado es **LogisticRegression** de scikit-learn. La persistencia se gestiona con:

- `models/model_active.joblib` – modelo activo serializado
- `models/accumulated_data.joblib` – dataset acumulado entre entrenamientos
- `models/training_history.json` – registro de todas las versiones

3 Flujo de Continuous Training

3.1 Bootstrap

Al arrancar el servidor por primera vez (o tras un reset), se entrena automáticamente un modelo base (v1.0-base) usando el dataset Iris completo de scikit-learn (150 muestras, `split 80/20`, `random_state=42`, `stratify=y`). Este modelo obtiene una accuracy de **0.9667** y queda registrado en el historial.

3.2 Ciclo de reentrenamiento

Cada llamada a POST `/train` sigue estos pasos:

1. Se reciben las nuevas muestras etiquetadas (mínimo 5).
2. Se construye el dataset de entrenamiento:
 - **Incremental** (`retrain_from_scratch=false`): se concatenan las nuevas muestras con las acumuladas en entrenamientos anteriores.
 - **Desde cero** (`retrain_from_scratch=true`): solo se usan las nuevas muestras.
3. Se entrena un nuevo modelo **LogisticRegression**.
4. Se evalúa el modelo:
 - Si hay ≥ 20 muestras: se usa un `split 80/20` para validación.
 - Si hay < 20 muestras: se evalúa sobre el propio conjunto de entrenamiento (con aviso).
5. Se aplica el **quality gate** según la política configurada.
6. Si pasa el gate: el nuevo modelo reemplaza al activo.
7. El resultado siempre queda registrado en el historial, independientemente de si se activa.

4 Razonamiento previo: elección de política por escenario

Antes de implementar las políticas, se analizan tres escenarios hipotéticos para justificar qué política sería más adecuada en cada caso.

4.1 Escenario 1: Detección de fraude bancario

Política recomendada: `per_class_f1` (clase positiva = fraude)

En fraude bancario, los falsos negativos (fraude no detectado) son mucho más costosos que los falsos positivos (transacción legítima bloqueada). La accuracy global puede ser alta simplemente prediciendo “no fraude” para todo, ya que los casos de fraude son raros. Lo que importa es el F1-score de la clase “fraude”: si el nuevo modelo no mejora la detección de fraude, no debe activarse aunque su accuracy global sea superior.

4.2 Escenario 2: Recomendación de películas

Política recomendada: `any_improvement`

El coste de un error es bajo (recomendar una película irrelevante no causa daño). Cualquier mejora marginal en accuracy es bienvenida porque incluso pequeñas mejoras se traducen en mejor experiencia de usuario a escala. No tiene sentido exigir un delta mínimo: si el modelo nuevo es al menos tan bueno como el anterior, vale la pena activarlo porque puede haber aprendido patrones más recientes.

4.3 Escenario 3: Diagnóstico médico

Política recomendada: `min_delta` (con delta significativo, ej. 0.02–0.05)

En diagnóstico médico, la estabilidad es crítica. Los profesionales de salud confían en que el modelo se comporta de forma predecible. Activar un modelo nuevo por una mejora de 0.001 en accuracy no justifica el riesgo de cambios imprevistos en el comportamiento. Se necesita una mejora demostrable y significativa para justificar la transición, de ahí el uso de `min_delta` con un umbral conservador.

4.4 Diseño previo en pseudocódigo

```
enum ActivationPolicyType:
    ANY_IMPROVEMENT    -- accuracy_new >= accuracy_previous
    MIN_DELTA           -- accuracy_new >= accuracy_previous + delta
    PER_CLASS_F1        -- f1_new[clase] >= f1_prev[clase]

class ActivationPolicy:
    type: ActivationPolicyType (default: ANY_IMPROVEMENT)
    min_delta: float            (default: 0.01, solo para MIN_DELTA)
    target_class: int           (default: 0, solo para PER_CLASS_F1)

function evaluate_quality_gate(policy, accuracy_new, accuracy_prev,
                               y_val, y_pred_new, y_pred_prev):
    if accuracy_prev is None:
        return True, "Primer modelo"

    switch policy.type:
```

```
ANY_IMPROVEMENT: return accuracy_new >= accuracy_prev
MIN_DELTA:       return accuracy_new >= accuracy_prev + policy.min_delta
PER_CLASS_F1:    return f1(y_val, y_pred_new)[clase] >= f1(y_val, y_pred_prev)[clase]
```

Este diseño se implementó en `main.py` con las clases `ActivationPolicyType`, `ActivationPolicy` y la función `evaluate_quality_gate()`.

5 Políticas de activación (ejercicio de programación)

Se han implementado tres políticas de activación configurables en el endpoint `/train` mediante el campo `activation_policy`.

5.1 any_improvement (por defecto)

Activa el modelo si su accuracy es mayor o igual a la del modelo anterior:

```
accuracy_new >= accuracy_previous -> ACTIVAR
accuracy_new <  accuracy_previous -> RECHAZAR
```

Es la política más permisiva. Permite activaciones laterales (mismo accuracy).

5.2 min_delta

Exige que el nuevo modelo mejore al menos un `min_delta` respecto al anterior:

```
accuracy_new >= accuracy_previous + min_delta -> ACTIVAR
```

Evita activaciones innecesarias cuando la mejora es marginal. Útil en producción para evitar el “churn” de modelos que no aportan mejoras reales.

Ejemplo de uso:

```
{
  "activation_policy": {
    "type": "min_delta",
    "min_delta": 0.05
  }
}
```

5.3 per_class_f1

Activa el modelo si el F1-score de una clase específica mejora respecto al modelo anterior, evaluando ambos modelos sobre el mismo conjunto de validación:

```
F1_clase_nueva >= F1_clase_anterior -> ACTIVAR
```

Útil cuando una clase es más crítica que las demás (ej. detectar una enfermedad rara, o una especie en peligro de extinción).

Ejemplo de uso:

```
{
  "activation_policy": {
    "type": "per_class_f1",
    "target_class": 1
  }
}
```

Todas las políticas son **retrocompatibles**: una llamada a `/train` sin el campo `activation_policy` usa `any_improvement` por defecto.

6 Ejercicio actual de la entrega LAB3

6.1 Ejecución del flujo completo

Se ejecutó el script `demo_ct.py -reset` con el servidor en marcha. El flujo completo realizó los siguientes pasos y produjo el historial:

Versión	Status	Accuracy	Política	Razón
v1.0-base	activado	0.9667	-	Bootstrap inicial con dataset Iris completo
v2.0	activado	1.0000	any_improvement	$1.0000 \geq 0.9667$
v3.0	rechazado	0.6000	any_improvement	$0.6000 < 1.0000$
v4.0	activado	1.0000	min_delta	$1.0000 \geq 0.6500$ (= 0.60 + delta 0.05)
v5.0	activado	1.0000	per_class_f1	F1 versicolor: nuevo=1.0000 vs anterior=1.0000

El modelo rechazado (v3.0) fue entrenado con 10 muestras cuyas etiquetas eran deliberadamente incorrectas (setosa etiquetada como virginica, etc.), lo que produjo un modelo con accuracy de 0.60, muy inferior al 1.00 del modelo activo.

6.2 Análisis del historial

El fichero `training_history.json` registra **5 versiones**: 4 activadas y 1 rechazada.

Observaciones:

- **El historial registra siempre todos los intentos**, incluso los rechazados. Esto es fundamental para la auditabilidad del sistema: permite saber qué datos llegaron, qué modelo produjeron, y por qué fue descartado.
- **La versión rechazada (v3.0)** fue entrenada desde cero (`retrain_from_scratch=true`) con solo 10 muestras ruidosas, sin datos acumulados. El modelo resultante fue muy deficiente porque los datos de entrenamiento eran incorrectos.
- **El campo `gate_reason`** registra la razón exacta de la decisión del quality gate, lo que facilita la depuración cuando un modelo inesperadamente es rechazado o activado.
- **Cada entrada registra la política usada**, permitiendo auditar no solo el resultado sino también el criterio de evaluación aplicado en cada momento.

6.3 Incremental vs. desde cero

Se realizaron dos llamadas a `/train` con las mismas 10 muestras correctamente etiquetadas.

6.3.1 Resultados obtenidos

Modo	Accuracy	Activado	Muestras usadas
Incremental (<code>retrain_from_scratch=false</code>)	1.0000	Sí	~ 70 (10 nuevas + ~ 60 acumuladas)
Desde cero (<code>retrain_from_scratch=true</code>)	1.0000	Sí	10 (solo las enviadas)

En este caso concreto ambos modos obtuvieron la misma accuracy (1.0000) porque las muestras son perfectamente representativas y el dataset Iris es sencillo. Sin embargo, las diferencias conceptuales son importantes.

Incremental (`retrain_from_scratch=false`)

- El modelo se entrena con las nuevas muestras + los datos acumulados de entrenamientos anteriores.
- Mayor cantidad de datos → modelo más robusto y generalizable.
- Si hay ≥ 20 muestras totales, se usa un split 80/20 para validación real.
- Accuracy típicamente más alta y más fiable.

Desde cero (`retrain_from_scratch=true`)

- El modelo se entrena solo con las muestras enviadas en la petición.
- Con < 20 muestras, la evaluación se hace sobre el propio conjunto de entrenamiento (sin split de validación), lo que puede sobreestimar la accuracy real.
- El modelo es más sensible a la calidad y cantidad de las muestras enviadas.
- Útil cuando se quiere “olvidar” datos anteriores que puedan estar obsoletos o ser erróneos.

Conclusión El entrenamiento incremental es más estable porque se beneficia de todos los datos históricos. El entrenamiento desde cero es útil cuando se detecta un cambio de distribución en los datos (*data drift*) y los datos antiguos ya no son representativos.

7 Preguntas de reflexión

7.1 Reproducibilidad: si se borra `models/` y se reinicia el servidor, ¿se obtiene el mismo modelo?

Sí. El bootstrap usa `random_state=42` en el `train_test_split` (línea 191) y en el `LogisticRegression` (línea 193), y aplica estratificación (`stratify=y`, línea 191) para garantizar proporciones iguales de cada clase. Con los mismos datos de entrada (el dataset Iris de scikit-learn, que es determinístico) y las mismas semillas, el resultado es idéntico en cada ejecución: mismo modelo, mismo accuracy (0.9667), misma versión (v1.0-base).

El sistema es por tanto **reproducible** en su estado inicial. Los modelos posteriores dependen de los datos que se envíen vía `/train`, que son externos al sistema.

7.2 Edge case: ¿qué ocurre si todas las muestras enviadas son de la misma clase?

El código lo detecta explícitamente (líneas 247–251 de `main.py`):

```
if len(np.unique(y_train)) < 2:
    raise HTTPException(
        status_code=422,
        detail="El dataset de entrenamiento debe contener al menos 2 clases distintas."
    )
```

Paso a paso, si se envían 10 muestras perfectas pero todas de setosa (clase 0):

1. Se reciben las 10 muestras (pasan la validación de mínimo 5).
2. Se construye el dataset:
 - Si `retrain_from_scratch=true`: solo las 10 muestras de setosa.
 - Si `retrain_from_scratch=false` y hay datos acumulados con otras clases: se mezclan.
3. Se comprueba el número de clases distintas:
 - En el caso `from_scratch`: solo hay clase 0 $\rightarrow$ `np.unique(y_train)` devuelve `[0]` $\rightarrow$ `len < 2` $\rightarrow$ **HTTP 422, el modelo NO se entrena.**
 - En el caso incremental con datos previos de 3 clases: hay ≥ 2 clases $\rightarrow$ se entrena, pero el modelo estará sesgado hacia setosa.

La restricción tiene sentido porque un clasificador entrenado con una sola clase no puede aprender a distinguir entre clases.

7.3 ¿Por qué registrar los modelos rechazados en el historial?

Por tres razones fundamentales:

- **Auditabilidad:** permite saber exactamente qué datos llegaron al sistema, cuándo, y qué modelo produjeron, aunque ese modelo nunca se activara.
- **Depuración de datos:** si un lote de datos produce un modelo muy malo, el registro permite identificar el problema (cuándo llegó, cuántas muestras había, qué accuracy obtuvo) y correlacionarlo con posibles errores en el pipeline de datos.
- **Trazabilidad regulatoria:** en entornos donde el modelo toma decisiones críticas (sanidad, finanzas), es obligatorio justificar por qué ciertos modelos fueron descartados. El historial completo es la evidencia de ese proceso de decisión.

7.4 Quality gate: $\geq$ o $>$?

$\geq$ es más adecuado para producción, por las siguientes razones:

- Con $>$, un modelo que obtiene exactamente la misma accuracy que el anterior nunca se activaría, aunque haya sido entrenado con datos más recientes. Esto bloquea innecesariamente la actualización del modelo.
- Con $\geq$, un modelo con igual accuracy puede activarse. Esto es válido porque el nuevo modelo puede haber aprendido patrones más recientes o ser más representativo de la distribución actual de datos, aunque numéricamente tenga la misma accuracy.

- En la práctica, cuando el dataset crece con muestras de buena calidad, la accuracy se estabiliza cerca del máximo (ej. 0.9667 o 1.0000). Con $>$ estricto, el sistema quedaría “bloqueado”: ninguna actualización pasaría el gate aunque los datos mejoren. Con $>=$, el modelo puede actualizarse incorporando datos recientes.
- El problema de $>=$ es el “churn”: modelos que se activan sin mejora real, generando inestabilidad en producción. La política `min_delta` resuelve este problema exigiendo una mejora mínima configurable, combinando lo mejor de ambos enfoques.

En resumen: $>=$ para el caso general, `min_delta` cuando se quiere evitar activaciones laterales en producción.

